

Training recipes and graph-native learning

ar088 · 14 August 2026 · Draft

Training is separate from the graph

`TrainSpec` describes standard learning choices without embedding a training loop in graph structure. It refers to named outputs and parameters in one network.

```
from tools.snnlang import training

recipe = training.TrainSpec(
    objectives=[training.CrossEntropy(
        prediction=readout,
        target="digit",
    )],
    parameter_groups=[training.ParameterGroup(
        name="feedforward",
        parameters=readout.parameters,
        lr=1e-3,
    )],
    optimizer=training.AdamW(weight_decay=1e-4),
    epochs=20,
)

bundle = snn.compile(net, training=recipe)
```

The recipe vocabulary includes objectives, named parameter groups, learning rates, frozen parameters, optimizer settings, regularizers, stop-gradient boundaries, epoch count, gradient clipping, and surrogate configuration.

Current boundary

Training recipes compile, and a narrow one-layer MNIST PING recipe can be translated into the legacy trainer. Graph-native training is not implemented. A graph training request fails explicitly and never falls back silently to legacy execution.

The complete implementation also needs deterministic initialization, differentiable-reachability checks, recurrent trainability, variable-rate training, spike-budget regularization, selected and final checkpoints, optimizer-state replay, exact resume, and layered legacy-versus-graph conformance tests.

Next: Runtime state, checkpoints, and provenance